

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

MIPS32 Assignment 2

July 2026

Instructions: *This is an individual assignment. Each student must demonstrate the working program to a Teaching Assistant for evaluation. Submit a single zipped folder named `MIPSAssignment2_{Your_Roll_No}.zip`. The folder must contain the complete and well-commented MIPS32 assembly source file. The source file must include a header containing the student's name, roll number, assignment number, and date of submission.*

Least Common Multiple Using Repeated Addition

1. Objective

The objective of this assignment is to write a MIPS32 assembly-language program that calculates the Least Common Multiple (LCM) of two positive integers.

The LCM must be calculated using repeated addition, comparison branches, labels, and a loop. Multiplication, division, and remainder instructions must not be used.

Through this assignment, students will practise:

1. reading integer inputs using system calls;
2. storing and manipulating values using registers;
3. comparing integer values using branch instructions;
4. constructing a loop by branching back to an earlier label;
5. preserving original input values while modifying temporary values;
6. validating user input; and
7. printing integer results and error messages using system calls.

2. Background

The Least Common Multiple of two positive integers is the smallest positive integer that is exactly divisible by both numbers.

For example, the positive multiples of 6 are

6, 12, 18, 24, 30, 36, ...

and the positive multiples of 8 are

$$8, 16, 24, 32, 40, \dots$$

The smallest value appearing in both sequences is 24. Therefore,

$$\text{LCM}(6, 8) = 24.$$

Let the two input numbers be denoted by A and B . Two running multiples, M_A and M_B , are initialized as

$$M_A = A \quad \text{and} \quad M_B = B.$$

The smaller running multiple is repeatedly increased by its corresponding original input value. The process stops when both running multiples become equal.

3. Algorithm

Algorithm 1 LCM using repeated addition

Require: Two positive integers A and B

Ensure: The Least Common Multiple of A and B

```
1:  $M_A \leftarrow A$ 
2:  $M_B \leftarrow B$ 
3: while  $M_A \neq M_B$  do
4:   if  $M_A < M_B$  then
5:      $M_A \leftarrow M_A + A$ 
6:   else
7:      $M_B \leftarrow M_B + B$ 
8: return  $M_A$ 
```

For example, when $A = 6$ and $B = 8$, the values evolve as follows:

M_A	M_B
6	8
12	8
12	16
18	16
18	24
24	24

Hence,

$$\text{LCM}(6, 8) = 24.$$

4. Assignment Task

Write a MIPS32 assembly-language program that performs the following operations:

1. Ask the user to enter the first positive integer.
2. Read the first integer using syscall 5 and store it in `$t0`.
3. Ask the user to enter the second positive integer.
4. Read the second integer using syscall 5 and store it in `$t1`.

5. Verify that both input values are strictly positive.
6. If either input is zero or negative, print an error message and terminate the program.
7. Preserve the original input values throughout the LCM loop.
8. Copy the first input into **\$t2**. This register will hold the first running multiple.
9. Copy the second input into **\$t3**. This register will hold the second running multiple.
10. Compare the two running multiples.
11. If the running multiples are equal, terminate the loop.
12. If the first running multiple is smaller, add the original first input to it.
13. If the second running multiple is smaller, add the original second input to it.
14. Branch back to the comparison label after each addition.
15. Print the common value as the LCM.
16. Terminate the program using syscall 10.

5. Register-Usage Guidelines

The following register allocation must be used:

Register	Required purpose
\$t0	Original first input A
\$t1	Original second input B
\$t2	Running multiple M_A
\$t3	Running multiple M_B
\$v0	Syscall number and returned integer input
\$a0	Address of a string or integer to be printed

Registers **\$t0** and **\$t1** must not be modified after input validation because they contain the original input values.

When the first running multiple is smaller, the required operation is

$$\text{\$t2} \leftarrow \text{\$t2} + \text{\$t0}.$$

When the second running multiple is smaller, the required operation is

$$\text{\$t3} \leftarrow \text{\$t3} + \text{\$t1}.$$

6. Input Validation

The program must accept only positive integers. The following inputs are invalid:

- zero;
- a negative first input; and
- a negative second input.

Students may use:

- `beq` to check whether a value is zero; and
- `blt` or `bltz` to check whether a value is negative.

For invalid input, the program must print a message similar to:

Error: Both inputs must be positive integers.

After printing the error message, the program must terminate using `syscall 10`.

7. Allowed Instructions and Directives

The solution should primarily use the following MIPS32 assembler directives and instructions:

- `.data`, `.text`, `.globl`, and `.ascii`;
- `la`, `li`, and `move`;
- `add` and `sub`;
- `beq`, `bne`, `bgt`, `blt`, and `bltz`;
- `b`; and
- `syscall`.

Pseudo-instructions supported by MARS or QtSPIM may be used.

8. Restrictions

The following instructions or methods must not be used:

- `mul`;
- `mult`;
- `multu`;
- `div`;
- `rem`;
- the formula

$$\text{LCM}(A, B) = \frac{A \times B}{\text{GCD}(A, B)};$$

- a lookup table containing precomputed LCM values; and
- hard-coded answers for individual test cases.

The LCM must be obtained only by generating and comparing multiples through repeated addition.

9. Suggested Program Structure

Students may organize their program using labels similar to the following:

```
.data
    # Input prompts
    # Output message
    # Invalid-input message
    # Newline string

.text
.globl main

main:
    # Read the first input
    # Read the second input
    # Validate both inputs
    # Initialize the running multiples

lcm_loop:
    # Check whether the running multiples are equal
    # Compare the running multiples
    # Branch to the appropriate addition operation

increase_first_multiple:
    # Add the original first input
    # Branch back to lcm_loop

increase_second_multiple:
    # Add the original second input
    # Branch back to lcm_loop

print_lcm:
    # Print the result
    # Terminate the program

invalid_input:
    # Print the error message
    # Terminate the program
```

The listing above provides only the program structure. Students must complete the MIPS32 instructions independently.

10. System-Call Guidelines

Service	Value in \$v0	Argument or result
Print integer	1	Integer must be placed in \$a0
Print string	4	Address of the string must be placed in \$a0
Read integer	5	Input integer is returned in \$v0
Exit program	10	No argument is required

After syscall 5, the input value must be copied from \$v0 into the required temporary register using move.

11. Sample Runs

Sample Run 1

```
Enter the first positive integer: 6
Enter the second positive integer: 8
The LCM of the two numbers is: 24
```

Sample Run 2

```
Enter the first positive integer: 7
Enter the second positive integer: 5
The LCM of the two numbers is: 35
```

Sample Run 3

```
Enter the first positive integer: 12
Enter the second positive integer: 12
The LCM of the two numbers is: 12
```

Sample Run 4: Invalid Input

```
Enter the first positive integer: 0
Enter the second positive integer: 8
Error: Both inputs must be positive integers.
```

12. Required Test Cases

First input	Second input	Expected result
6	8	24
7	5	35
12	12	12
1	15	15
4	10	20
9	6	18
13	17	221
0	8	Error message
-4	10	Error message
8	-6	Error message

Additional test cases may be used during evaluation.

13. Correctness Requirements

A correct implementation must satisfy all the following conditions:

1. Both inputs are read from the user.
2. Invalid inputs are detected before the LCM loop begins.
3. The original input values remain unchanged during the loop.
4. The two running multiples are initialized correctly.
5. The loop terminates immediately when the inputs are equal.

6. Only the smaller running multiple is increased in each iteration.
7. The correct original input is added to its corresponding running multiple.
8. The equality condition is checked during every iteration.
9. The common value is printed as the LCM.
10. Multiplication, division, and remainder instructions are not used.
11. The program exits using syscall 10.

14. Common Errors to Avoid

1. Modifying `$t0` or `$t1` inside the loop.
2. Adding the second input to the first running multiple.
3. Adding the first input to the second running multiple.
4. Increasing the larger running multiple instead of the smaller one.
5. Failing to check whether the two running multiples are equal.
6. Branching to the wrong label after an addition.
7. Forgetting to branch back to the beginning of the loop.
8. Entering an infinite loop because zero was not rejected.
9. Printing an original input instead of the final common multiple.
10. Omitting syscall 10.

15. Submission Requirements

Submit a zipped folder named

`MIPSAssignment2_{Your_Roll_No}.zip`

containing the source file

`lcm_{Your_Roll_No}.S.`

The source file must begin with a comment header similar to the following:

```
# -----
# Name:
# Roll Number:
# Course: Computer Organization and Architecture Laboratory
# Assignment: MIPS32 Assignment 2
# Problem: LCM using repeated addition
# Date:
# -----
```

The source code must contain meaningful comments explaining the purpose of each register, input validation, initialization, loop condition, comparison branches, repeated-addition operations, and result-printing section.